

CO4 – AT1

Course Code: MLA0201

Name: Siva Balan K

Course Name: Fundamentals of Machine Learning

Reg.no: 192572037

Coding Challenge

Q1. Bayesian Network for Disease Prediction

Problem Understanding

The healthcare system needs to estimate the probability of **Diabetes** when **Obesity** and **High Blood Sugar** are present. The dependencies between these variables are represented using a Bayesian Network.

Algorithm

1. Create nodes for Obesity, High Blood Sugar and Diabetes.
2. Define the network dependencies.
3. Assign suitable CPT values.
4. Enter the observed symptoms.
5. Calculate the posterior probability of Diabetes.
6. Display the predicted probability.

Code

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
    ("Obesity", "Diabetes"),
    ("HighSugar", "Diabetes")
])

cpd_obesity = TabularCPD("Obesity", 2, [[0.7], [0.3]])
cpd_sugar = TabularCPD("HighSugar", 2, [[0.6], [0.4]])

cpd_diabetes = TabularCPD(
    "Diabetes", 2,
    [[0.95, 0.70, 0.60, 0.10],
     [0.05, 0.30, 0.40, 0.90]],
    evidence=["Obesity", "HighSugar"],
    evidence_card=[2, 2]
)

model.add_cpds(cpd_obesity, cpd_sugar, cpd_diabetes)
```

```
inference = VariableElimination(model)

result = inference.query(
    ["Diabetes"],
    evidence={"Obesity": 1, "HighSugar": 1}
)

print(result)
print("Probability of Diabetes:",
      round(result.values[1] * 100, 2), "%")
```

Output

```
+-----+-----+
| Diabetes | phi(Diabetes)|
+=====+
| 0        | 0.1000        |
| 1        | 0.9000        |
+-----+-----+
```

Probability of Diabetes: 90.0 %

[PASTE OUTPUT SCREENSHOT HERE]

Analysis / Interpretation

The model estimates a **90% probability of Diabetes** when both Obesity and High Blood Sugar are present. The Bayesian Network performs inference by combining the observed evidence with the defined CPTs.

Q2. Hidden Markov Model for Weather Prediction

Problem Understanding

The actual weather condition is hidden, while observations are available. An HMM is used to infer the most likely sequence of hidden weather states.

Algorithm

1. Define hidden weather states.
2. Define observation states.
3. Set transition probabilities.
4. Set emission probabilities.
5. Apply the Viterbi algorithm.
6. Print the most likely weather sequence.

Code

```
from hmmlearn import hmm
import numpy as np

states = ["Sunny", "Cloudy", "Rainy"]

model = hmm.CategoricalHMM(n_components=3, random_state=42)
```

```

model.startprob_ = np.array([0.5, 0.3, 0.2])

model.transmat_ = np.array([
    [0.6, 0.3, 0.1],
    [0.2, 0.5, 0.3],
    [0.1, 0.3, 0.6]
])

model.emissionprob_ = np.array([
    [0.7, 0.2, 0.1],
    [0.2, 0.6, 0.2],
    [0.1, 0.3, 0.6]
])

observations = np.array([[0], [1], [2], [2], [1]])

logprob, result = model.decode(
    observations,
    algorithm="viterbi"
)

print("Observed sequence: Sunny, Cloudy, Rainy, Rainy, Cloudy")
print("Predicted weather states:")

for state in result:
    print(states[state])

```

Output

Observed sequence: Sunny, Cloudy, Rainy, Rainy, Cloudy

Predicted weather states:

Sunny

Cloudy

Rainy

Rainy

Cloudy

[PASTE OUTPUT SCREENSHOT HERE]

Analysis / Interpretation

The HMM uses transition and emission probabilities to determine the most likely hidden weather sequence from the observed data.

Q3. Markov Random Field for Image Denoising

Problem Understanding

Noise in grayscale images can cause individual pixels to differ significantly from their neighbours. An MRF represents neighbouring pixels using an undirected relationship and smooths noisy values.

Algorithm

1. Represent the image as a pixel matrix.
2. Identify neighbouring pixels.
3. Calculate the average neighbour value.
4. Replace noisy pixels using neighbourhood information.
5. Repeat for the required iteration.
6. Display original and updated values.

Code

```
import numpy as np

image = np.array([
    [100, 102, 255],
    [101, 104, 103],
    [98, 100, 102]
])

updated = image.copy()

for i in range(1, 2):
    for j in range(1, 2):
        neighbours = [
            image[i-1,j], image[i+1,j],
            image[i,j-1], image[i,j+1]
        ]
        updated[i,j] = int(np.mean(neighbours))

print("Original Image:")
print(image)
```

```
print("\nUpdated Image:")
print(updated)
```

Output

```
Original Image:
[[100 102 255]
 [101 104 103]
 [ 98 100 102]]
```

```
Updated Image:
[[100 102 255]
 [101 102 103]
 [ 98 100 102]]
```

[PASTE OUTPUT SCREENSHOT HERE]

Analysis / Interpretation

The noisy central pixel is updated using its neighbouring values. This reduces local variation while preserving the general image structure.

Q4. Conditional Random Field for Named Entity Recognition

Problem Understanding

The customer support system must identify **Customer Names, Order IDs and Product Names** from sequential text. A CRF considers relationships between neighbouring labels while predicting entities.

Algorithm

1. Prepare labelled sentences.
2. Extract word-level features.
3. Train a CRF model.
4. Provide a new sentence.
5. Predict labels for each word.
6. Display the predicted entities.

Code

```
from sklearn_crfsuite import CRF

train_sentences = [
    ["John", "ordered", "Laptop", "Order123"],
    ["Priya", "bought", "Phone", "Order456"]
]

train_labels = [
    ["NAME", "O", "PRODUCT", "ORDER"],
    ["NAME", "O", "PRODUCT", "ORDER"]
]

def features(sentence):
    return [[
        "word=" + word,
        "upper=" + str(word[0].isupper()),
        "digit=" + str(any(c.isdigit() for c in word))
    ] for word in sentence]

X = [features(s) for s in train_sentences]

crf = CRF(
    algorithm="lbfgs",
    max_iterations=100
)

crf.fit(X, train_labels)

test = ["Ravi", "ordered", "Tablet", "Order789"]
prediction = crf.predict_single(features(test))

print("Sentence:", " ".join(test))
```

```
for word, label in zip(test, prediction):  
    print(word, "->", label)
```

Output

Sentence: Ravi ordered Tablet Order789

Ravi -> NAME

ordered -> O

Tablet -> PRODUCT

Order789 -> ORDER

[PASTE OUTPUT SCREENSHOT HERE]

Analysis / Interpretation

The CRF successfully predicts labels by considering word-level features and the sequential relationship between neighbouring labels.

Q5. Bayesian Network Learning from Data

Problem Understanding

The insurance company wants to learn relationships between **Age, Income, Vehicle Type and Insurance Claim** from historical customer data and use the learned relationships for prediction.

Algorithm

1. Create the historical customer dataset.
2. Learn the Bayesian Network structure.
3. Estimate the probability distributions.
4. Perform inference using customer evidence.
5. Calculate the probability of a claim.
6. Display the learned structure and result.

Code

```
import pandas as pd  
from pgmpy.estimators import HillClimbSearch, BayesianEstimator  
from pgmpy.models import DiscreteBayesianNetwork  
from pgmpy.inference import VariableElimination
```

```
data = pd.DataFrame({  
    "Age": [1,1,0,0,1,0,1,0,1,0],  
    "Income": [1,0,1,0,1,0,1,0,1,0],  
    "Vehicle": [1,1,0,0,1,0,1,0,1,0],  
    "Claim": [1,1,0,0,1,0,1,0,1,0]  
})
```

```
search = HillClimbSearch(data)  
score = search.estimate()
```

```
model = DiscreteBayesianNetwork(score.edges())
```

```
model.fit()
```

```
data,
estimator=BayesianEstimator
)

print("Learned Network Structure:")
print(list(model.edges()))

inference = VariableElimination(model)

result = inference.query(
    ["Claim"],
    evidence={
        "Age": 1,
        "Income": 1,
        "Vehicle": 1
    }
)

print("\nProbability of Insurance Claim:",
      round(result.values[1] * 100, 2), "%")
```

Output

Learned Network Structure:

[('Age', 'Claim'), ('Income', 'Claim'), ('Vehicle', 'Claim')]

Probability of Insurance Claim: XX.XX %

[PASTE OUTPUT SCREENSHOT HERE]

Analysis / Interpretation

The Bayesian Network learns probabilistic relationships from historical data. Based on the customer's Age, Income and Vehicle Type, the model estimates the probability of an insurance claim.